

LAPORAN SOFTWARE QUALITY ASSURANCE METODE WHITEBOX – CODE WALKTHROUGH



Anggota kelompok:

20231310046
20231310047

M. Irvan Alfiansyah
M. Nur Yanfa

**PROGRAM STUDI TEKNIK INFORMATIKA
UNIVERSITAS KEBANGSAAN REPUBLIK INDONESIA**

*Jln. Terusan Halimun No. 37 (Pelajar Pejuang 45) Lingkar Selatan Kec. Lengkong
kota Bandung Jawa Barat 40263*

1. DASAR TEORI CODE WALKTHROUGH

Code Walkthrough adalah metode white box testing yang dilakukan dengan menelusuri source code secara sistematis dari titik masuk hingga titik keluar program. Penguji membaca alur kode, menandai keputusan penting, memeriksa aliran data, lalu membandingkan implementasi dengan kebutuhan sistem.

Pada fitur Checkout ACID, metode ini digunakan karena proses checkout tidak hanya membuat order, melainkan juga mencakup:

1. Validasi sesi pengguna (autentikasi JWT)
2. Pembacaan keranjang belanja (cart)
3. Pembentukan transaksi database (BEGIN / COMMIT / ROLLBACK)
4. Validasi stok produk dan varian
5. Pengurangan stok secara atomik
6. Pembuatan order dan order item
7. Penghapusan cart setelah order berhasil
8. Pembuatan Snap Token Midtrans (payment gateway)
9. Pengembalian respons HTTP yang sesuai

Jika salah satu tahap tersebut tidak sesuai, dapat terjadi transaksi parsial, stok minus, order tanpa item, atau respons HTTP yang keliru. Oleh karena itu, code walkthrough dilakukan menyeluruh pada ketiga layer: Handler, Usecase, dan Repository.

2. RUANG LINGKUP PENGUJIAN

Pengujian difokuskan pada fitur Checkout ACID yang sama dengan pengujian black-box sebelumnya. Tidak mencakup landing page, katalog, login, register, atau fitur non-checkout.

ID Black-Box	Skenario	Dicakup	Titik Kode yang Ditelusuri
CHK-01	Checkout berhasil	Ya	Handler, usecase, repository transaction
CHK-02	Stok tidak mencukupi	Ya	Validasi stok produk dan varian
CHK-03	Race condition checkout bersamaan	Ya	Locking FOR UPDATE dalam transaksi
CHK-04	Keranjang kosong	Ya	Guard <code>len(cartItems) == 0</code>
CHK-05	Midtrans API error	Ya	<code>snapTokenFn</code> dan fallback Midtrans error
CHK-06	Checkout tanpa JWT/session	Ya	Validasi <code>user_id</code> di handler
CHK-07	Atomicity stok dan order	Ya	<code>db.Transaction</code> dan rollback otomatis
CHK-08	Status order PENDING	Ya	Pembuatan <code>domain.Order</code> di repository

3. OBJEK SOURCE CODE YANG DIUJI

No	File	Baris	Fungsi / Bagian	Layer	Alasan Diuji
1	<i>order_handler.go</i>	15-26	classifyCheckoutError	HTTP helper	Pemetaan error bisnis & gateway ke status HTTP
2	<i>order_handler.go</i>	60-87	OrderHandler.Checkout	Handler	Titik masuk endpoint checkout
3	<i>order_usecase.go</i>	72-99	orderUsecase.Checkout	Usecase	Pusat orkestrasi cart, transaction, dan payment token
4	<i>order_repository.go</i>	23-120	CheckoutTransaction	Repository	Area utama transaksi ACID
5	<i>order_repository.go</i>	33-81	Loop cart item	Repository	Validasi stok, subtotal, dan order item
6	<i>order_repository.go</i>	85-109	Create order, items, delete cart	Repository	Bukti atomicity dan durability
7	<i>order_whitebox_additional_test.go</i>	110-259	Data flow & loop tests	Unit test	Bukti otomatis tambahan untuk hasil walkthrough

4. GAMBARAN ALUR CHECKOUT

Alur keseluruhan sistem checkout melewati tiga layer utama secara berurutan:

```
Client
-> POST /api/v1/orders/checkout
-> Auth middleware mengisi user_id
-> OrderHandler.Checkout
-> orderUsecase.Checkout
-> cartRepo.FindByUserID
-> orderRepo.CheckoutTransaction
```

```

-> PostgreSQL transaction (BEGIN)
    -> lock product FOR UPDATE
    -> lock variant FOR UPDATE (jika ada)
    -> validasi & kurangi stok
    -> create order + order_items
    -> delete cart
-> COMMIT
-> snapTokenFn / Midtrans (opsional)
-> response 201 Created atau error valid

```

Layer	Tanggung Jawab	Bukti Kode
Handler	Validasi sesi, membaca body opsional, mengubah error menjadi HTTP response	<i>OrderHandler.Checkout</i>
Usecase	Mengambil cart, menolak cart kosong, memanggil transaction, membuat token pembayaran	<i>orderUsecase.Checkout</i>
Repository	Menjalankan transaksi database atomik (lock, validasi stok, create, delete)	<i>CheckoutTransaction</i>

5. WALKTHROUGH FUNGSI: `classifyCheckoutError`

5.1 Source Code yang Diuji

```

func classifyCheckoutError(err error) *response.AppError {
    msg := err.Error()
    switch {
    case strings.Contains(msg, "keranjang belanja anda kosong"):
        return response.ErrBadRequest(msg)
    case strings.Contains(msg, "stok tidak mencukupi"),
        strings.Contains(msg, "Stok"),
        strings.Contains(msg, "stok"):
        return response.ErrBadRequest(msg)
    case strings.Contains(msg, "midtrans"),
        strings.Contains(msg, "Midtrans"),
        strings.Contains(msg, "snap"),
        strings.Contains(msg, "payment gateway"):
        return &response.AppError{
            Code: http.StatusBadGateway,
            Message: "Layanan pembayaran tidak tersedia",
            Detail: msg,
        }
    }
}

```

```

    }
    default:
        return response.ErrConflict(msg)
    }
}

```

5.2 Langkah Walkthrough

Langkah	Observasi Kode	Penilaian	Status
1	Error diubah menjadi string melalui <code>err.Error()</code>	Sesuai — sumber error dari repository/usecase berupa pesan bisnis	PASS
2	Pesan cart kosong dikembalikan sebagai 400 Bad Request	Sesuai CHK-04	PASS
3	Pesan stok kurang dikembalikan sebagai 400 Bad Request	Sesuai CHK-02 dan sample testing	PASS
4	Error Midtrans/snap/payment gateway dikembalikan sebagai 502 Bad Gateway	Sesuai CHK-05 — kegagalan layanan eksternal	PASS
5	Error lain dikembalikan sebagai Conflict (409)	Sesuai sebagai fallback non-server-crash	PASS

5.3 Analisis

Fungsi ini memperjelas batas antara error bisnis dan error server. Cart kosong atau stok kurang bukan kegagalan server sehingga status 400 lebih tepat daripada 500. Error Midtrans merupakan kegagalan layanan eksternal sehingga 502 lebih tepat daripada 500. Pemisahan ini konsisten dengan prinsip bahwa server tidak boleh mengembalikan 5xx untuk kondisi yang dapat diantisipasi.

6. WALKTHROUGH FUNGSI: OrderHandler.Checkout

6.1 Source Code yang Diuji

```

func (h *OrderHandler) Checkout(c *gin.Context) {
    uidVal, exists := c.Get("user_id")
    if !exists {
        response.Error(c, response.ErrUnauthorized("Sesi pengguna tidak valid"))
        return
    }
    uid, ok := uidVal.(string)
    if !ok {

```

```

        response.Error(c, response.ErrUnauthorized("Sesi pengguna tidak valid"))
        return
    }

    var req struct {
        VoucherCode string `json:"voucher_code"`
    }

    _ = c.ShouldBindJSON(&req)    // error diabaikan — body opsional

    order, err := h.orderUsecase.Checkout(uid, req.VoucherCode)
    if err != nil {
        response.Error(c, classifyCheckoutError(err))
        return
    }

    response.Success(c, http.StatusCreated,
        "Checkout berhasil. Silakan lakukan pembayaran.", order)
}

```

6.2 Langkah Walkthrough

Langkah	Baris	Kode / Kondisi	Hasil Pemeriksaan	Status
H-01	62-66	c.Get("user_id") tidak ada	Request tanpa JWT ditolak 401	PASS
H-02	67-71	user_id bukan string	Request ditolak sebagai sesi tidak valid	PASS
H-03	73-77	ShouldBindJSON tidak diwajibkan sukses (_ =)	Body kosong tetap valid — voucher opsional	PASS
H-04	79	h.orderUsecase.Checkout(uid, req.VoucherCode)	Handler tidak memproses bisnis checkout sendiri	PASS
H-05	80-83	Usecase error	Error diklasifikasi ke response tepat	PASS
H-06	86	Usecase sukses	Response 201 Created	PASS

6.3 Analisis Keamanan

Handler tidak mengambil user_id dari body request. Identitas pengguna berasal dari context yang diisi oleh middleware autentikasi JWT. Ini penting untuk mencegah pengguna melakukan checkout atas nama pengguna lain — sebuah celah keamanan yang umum ditemukan pada sistem yang tidak memvalidasi identitas dari sisi server.

7. WALKTHROUGH FUNGSI: orderUsecase.Checkout

7.1 Source Code yang Diuji

```
func (u *orderUsecase) Checkout(userID string, voucherCode string)
(*domain.Order, error) {

    cartItems, err := u.cartRepo.FindByUserID(userID)
    if err != nil {
        return nil, errors.New("gagal memuat keranjang belanja")
    }

    if len(cartItems) == 0 {
        return nil, errors.New("keranjang belanja anda kosong. tidak bisa
checkout")
    }

    order, err := u.orderRepo.CheckoutTransaction(userID, cartItems,
voucherCode)
    if err != nil {
        return nil, errors.New("Checkout gagal: " + err.Error())
    }

    snapResp, snapErr := u.snapTokenFn(order.ID, order.TotalAmount)
    if snapErr == nil && snapResp != nil {
        order.PaymentToken = &snapResp.Token
        order.PaymentURL = &snapResp.RedirectURL
        u.orderRepo.SavePaymentToken(order.ID, snapResp.Token,
snapResp.RedirectURL)
    } else if snapErr != nil {
        fmt.Printf("[MIDTRANS ERROR] Gagal generate Snap Token: %v\n", snapErr)
    }

    return order, nil
}
```

7.2 Langkah Walkthrough

Langkah	Baris	Observasi Kode	Dampak Terhadap Checkout ACID
U-01	73	Cart diambil berdasarkan userID	Isolation antar user terjaga — setiap checkout hanya membaca cart miliknya sendiri
U-02	74-76	Error cart repository menghentikan proses	Mencegah checkout dengan data cart tidak valid atau tidak terbaca
U-03	78-80	Cart kosong ditolak sebelum transaction	Tidak ada order kosong — sesuai CHK-04
U-04	82	Cart valid diteruskan ke repository transaction	Transaksi database dipusatkan di repository — usecase tidak menyentuh DB langsung
U-05	83-85	Transaction error dibungkus sebagai error checkout	Error stok/DB tidak menjadi success palsu
U-06	89	snapTokenFn(order.ID, order.TotalAmount)	Payment token memakai order dan amount hasil transaksi yang telah commit
U-07	90-93	Token sukses disimpan ke order dan database	Payment URL tersedia setelah checkout berhasil
U-08	94-96	Midtrans error hanya di-log, tidak di-return	Order internal tetap durable — sesuai CHK-05 dan CHK-07

7.3 Analisis

Usecase memisahkan dua jenis proses secara tegas:

- Proses internal wajib konsisten: cart menjadi order. Kegagalan di sini menyebabkan return error dan tidak ada order yang tersimpan.
- Proses eksternal dapat gagal: pembuatan Snap Token Midtrans. Kegagalan di sini hanya dicatat (log) dan tidak membatalkan order yang sudah tersimpan.

Desain ini sesuai dengan CHK-05: kegagalan Midtrans tidak boleh menyebabkan server crash atau rollback order yang sudah berhasil dibuat. Skenario ini juga telah divalidasi oleh unit test `TestCheckout_MidtransFails_OrderTetapKembali`.

8. WALKTHROUGH FUNGSI: CheckoutTransaction (Repository)

8.1 Source Code yang Diuji

```
err := r.db.Transaction(func(tx *gorm.DB) error {
    var totalAmount float64
    var orderItems []domain.OrderItem
    orderID := uuid.New().String()

    for _, item := range cartItems {
        var product domain.Product
        tx.Clauses(clause.Locking{Strength: "UPDATE"}).
            Where("id_product = ?", item.ProductID).
            First(&product) // lock FOR UPDATE

        priceAtPurchase := product.Price

        if item.VariantID != nil {
            var variant domain.ProductVariant
            tx.Clauses(clause.Locking{Strength: "UPDATE"}).
                Where("id_variant = ?", *item.VariantID).
                First(&variant) // lock variant FOR UPDATE

            if variant.Stock < item.Quantity {
                return fmt.Errorf("stok varian '%s' tidak mencukupi...",
                    variant.NameLabel, variant.Stock)
            }

            variant.Stock -= item.Quantity
            tx.Save(&variant)
            priceAtPurchase = variant.Price
        } else {
            if product.Stock < item.Quantity {
                return fmt.Errorf("stok produk '%s' tidak mencukupi...",
                    product.Name, product.Stock)
            }

            product.Stock -= item.Quantity
            tx.Save(&product)
        }

        totalAmount += priceAtPurchase * float64(item.Quantity)
        orderItems = append(orderItems, domain.OrderItem{ PriceAtPurchase:
            priceAtPurchase, ... })
    }
})
```

```

    }

    createdOrder = domain.Order{ Status: "PENDING", TotalAmount:
totalAmount, ... }
    tx.Create(&createdOrder)
    tx.Create(&orderItems)
    tx.Where("id_user = ?", userID).Delete(&domain.CartItem{})
    return nil // trigger COMMIT
})
// Jika return err sebelum nil -> GORM rollback otomatis

```

8.2 Langkah Walkthrough Repository

Langkah	Baris	Observasi Kode	Bukti ACID	Status
R-01	27	Seluruh proses berada dalam db.Transaction	Atomicity	PASS
R-02	31	orderId dibuat sekali sebelum loop	Konsistensi relasi order-item	PASS
R-03	34	Loop memproses seluruh cart item	Semua item masuk transaksi	PASS
R-04	37	Produk dikunci FOR UPDATE	Isolation saat concurrency	PASS
R-05	45	Varian juga dikunci FOR UPDATE	Isolation stok varian	PASS
R-06	48-54	Stok varian dicek sebelum dikurangi	Stok tidak negatif	PASS
R-07	58-65	Stok produk dicek sebelum dikurangi	Stok tidak negatif	PASS
R-08	69	totalAmount dihitung dari harga saat checkout	Consistency total pembayaran	PASS
R-09	71-80	Order item dibuat dengan PriceAtPurchase	Snapshot harga transaksi	PASS
R-10	86-95	Order dibuat dengan status PENDING	Sesuai CHK-08	PASS
R-11	97-104	Order dan order item disimpan ke database	Durability	PASS
R-12	107	Cart user dihapus dalam satu transaction	Atomic cleanup	PASS
R-13	115-117	Error menyebabkan rollback otomatis (GORM)	Atomicity — tidak ada partial commit	PASS

8.3 Analisis Repository

Repository adalah lapisan paling kritikal untuk menjamin ACID. Urutan eksekusi dalam satu transaksi adalah sebagai berikut:

```
lock product/variant (FOR UPDATE)
  -> cek stok (< quantity = error = rollback)
  -> kurangi stok
  -> hitung totalAmount
  -> append orderItems
-> create order (status: PENDING)
-> create order items (PriceAtPurchase)
-> delete cart items
-> return nil => COMMIT
```

Jika terjadi error sebelum return nil, GORM secara otomatis menjalankan ROLLBACK. Dengan demikian, order tidak akan tersimpan sebagian ketika stok tidak cukup atau query gagal di tengah proses.

9. CHECKLIST WALKTHROUGH

ID	Area	Pertanyaan Pemeriksaan	Hasil	Status
CW-01	Auth	Apakah checkout wajib menggunakan user terautentikasi?	user_id wajib ada di context	PASS
CW-02	Auth	Apakah user dapat menentukan user_id dari body?	Tidak — handler memakai context JWT	PASS
CW-03	Input	Apakah body checkout boleh kosong?	Ya — voucher opsional (ShouldBindJSON)	PASS
CW-04	Cart	Apakah cart kosong ditolak sebelum transaksi?	Ya — usecase cek len(cartItems) == 0	PASS
CW-05	Transaction	Apakah pengurangan stok dan pembuatan order atomik?	Ya — db.Transaction GORM	PASS
CW-06	Stok	Apakah stok dicek sebelum dikurangi?	Ya — produk dan varian divalidasi	PASS
CW-07	Stok	Apakah stok dilindungi dari race condition?	Ya — FOR UPDATE locking	PASS
CW-08	Order	Apakah order item menyimpan harga saat checkout?	Ya — PriceAtPurchase	PASS
CW-09	Order	Apakah status awal order benar?	Ya — Status: PENDING	PASS

ID	Area	Pertanyaan Pemeriksaan	Hasil	Status
CW-10	Cart	Apakah cart dihapus setelah order dibuat?	Ya — masih dalam transaction yang sama	PASS
CW-11	Error	Apakah error bisnis tidak menjadi HTTP 500?	Ya — dipetakan ke 400 / 409 / 502	PASS
CW-12	Payment	Apakah Midtrans error tidak menggagalkan order internal?	Ya — error di-log, order tetap di-return	PASS

Seluruh 12 checklist terpenuhi (12/12 PASS).

10. HUBUNGAN DENGAN PRINSIP ACID

Prinsip	Bukti Source Code	Penjelasan
Atomicity	<i>r.db.Transaction(...)</i> pada <i>repository</i>	Pengurangan stok, pembuatan order, order items, dan penghapusan cart terjadi sebagai satu unit — semua berhasil atau semua dibatalkan.
Consistency	Validasi stok sebelum pengurangan; <i>PriceAtPurchase</i> ; Status PENDING	Data order tidak dibuat jika stok tidak cukup. Harga di-snapshot saat transaksi sehingga tidak terpengaruh perubahan harga setelah checkout.
Isolation	<i>clause.Locking{Strength: "UPDATE"}</i>	Mencegah dua checkout bersamaan membaca dan mengurangi stok yang sama secara tidak terkendali — basis penanganan race condition (CHK-03).
Durability	<i>tx.Create(&createdOrder)</i> , <i>tx.Create(&orderItems)</i>	Setelah COMMIT sukses, order dan seluruh item tersimpan permanen di PostgreSQL dan tidak terpengaruh restart server.

11. TEST PENDUKUNG (UNIT TEST)

11.1 Command Pengujian

```
cd backend-go
go test ./internal/usecase -run "TestCheckout|TestDataFlow|TestLoopTesting" -v
```

11.2 Hasil Eksekusi

```
=== RUN    TestCheckout_CartRepoError
--- PASS: TestCheckout_CartRepoError
=== RUN    TestCheckout_EmptyCart
--- PASS: TestCheckout_EmptyCart
=== RUN    TestCheckout_TransactionFails_StokKurang
--- PASS: TestCheckout_TransactionFails_StokKurang
=== RUN    TestCheckout_Success
--- PASS: TestCheckout_Success
=== RUN    TestCheckout_MidtransFails_OrderTetapKembali
--- PASS: TestCheckout_MidtransFails_OrderTetapKembali
=== RUN    TestCheckout_SnapRespNil_OrderTetapKembali
--- PASS: TestCheckout_SnapRespNil_OrderTetapKembali
=== RUN    TestDataFlow_CheckoutCartToOrderAndPaymentToken
--- PASS: TestDataFlow_CheckoutCartToOrderAndPaymentToken
=== RUN    TestLoopTesting_CheckoutCartItemCounts
--- PASS: TestLoopTesting_CheckoutCartItemCounts
=== RUN    TestLoopTesting_BatchProcessSupplierOrders
--- PASS: TestLoopTesting_BatchProcessSupplierOrders
=== RUN    TestLoopTesting_BatchProcessSupplierOrders_NoValidOrder
--- PASS: TestLoopTesting_BatchProcessSupplierOrders_NoValidOrder
PASS
ok  github.com/nuryanfa/e-commerce-sqa/internal/usecase
```

11.3 Interpretasi Hasil

Nama Test	Skenario yang Divalidasi	Status
<i>TestCheckout_CartRepoError</i>	Error pada pembacaan cart repository	PASS
<i>TestCheckout_EmptyCart</i>	Cart kosong ditolak sebelum transaksi (CHK-04)	PASS
<i>TestCheckout_TransactionFails_StokKurang</i>	Stok tidak cukup, transaksi rollback (CHK-02)	PASS
<i>TestCheckout_Success</i>	Alur checkout normal end-to-end (CHK-01)	PASS
<i>TestCheckout_MidtransFails_OrderTetapKembali</i>	Midtrans error, order internal tetap tersimpan (CHK-05)	PASS

Nama Test	Skenario yang Divalidasi	Status
<i>TestCheckout_SnapRespNil_OrderTetapKembali</i>	Snap response nil, order tetap return	PASS
<i>TestDataFlow_CheckoutCartToOrderAndPaymentToken</i>	Alur data cart -> order -> payment token	PASS
<i>TestLoopTesting_CheckoutCartItemCounts</i>	Jumlah cart item bervariasi (loop testing)	PASS
<i>TestLoopTesting_BatchProcessSupplierOrders</i>	Batch pemrosesan order supplier valid	PASS
<i>TestLoopTesting_BatchProcessSupplierOrders_NoValidOrder</i>	Batch tanpa order valid — edge case loop	PASS

Seluruh 10 unit test lulus (10/10 PASS). Hasil ini memperkuat temuan walkthrough karena skenario utama telah diverifikasi secara otomatis dan dapat direproduksi.

12. TRACEABILITY MATRIX

Skenario Black-Box	Bukti Walkthrough	Test Pendukung	Kesimpulan
CHK-01 Checkout berhasil	Handler return 201, usecase return order, repository commit	<i>TestCheckout_Success</i>	Terpenuhi
CHK-02 Stok kurang	Repository return error sebelum order dibuat — rollback	<i>TestCheckout_TransactionFails_StokKurang</i>	Terpenuhi
CHK-03 Race condition	FOR UPDATE pada produk dan varian	Walkthrough repository (desain)	Terpenuhi secara desain
CHK-04 Cart kosong	Usecase menolak cart kosong sebelum transaksi	<i>TestCheckout_EmptyCart</i>	Terpenuhi
CHK-05 Midtrans error	snapErr di-log, order tetap return	<i>TestCheckout_MidtransFails_OrderTetapKembali</i>	Terpenuhi
CHK-06 Tanpa JWT	Handler menolak jika user_id tidak ada di context	Basic Path handler test	Terpenuhi

Skenario Black-Box	Bukti Walkthrough	Test Pendukung	Kesimpulan
CHK-07 Atomicity	db.Transaction dan rollback otomatis GORM	Walkthrough repository	Terpenuhi
CHK-08 Status PENDING	Status: PENDING saat create order	Walkthrough repository	Terpenuhi

13. TEMUAN DAN REKOMENDASI

ID	Temuan	Dampak	Rekomendasi	Prioritas
CW-F01	Checkout ACID sudah memakai transaction dan locking (FOR UPDATE)	Positif: stok terlindungi dari race condition	Dipertahankan dan jadikan standar pada modul lain	Tinggi
CW-F02	Error Midtrans tidak menggagalkan order internal	Positif: order tetap tersimpan meski payment gateway down	Tambahkan monitoring alert dan mekanisme retry payment token bila perlu	Sedang
CW-F03	Voucher masih diteruskan antar layer namun tidak aktif di repository	Dapat membingungkan dokumentasi dan pembaca kode	Jelaskan di SKPL/laporan bahwa voucher tidak termasuk scope checkout ACID aktif	Rendah
CW-F04	Mapping error berbasis string matching (strings.Contains)	Rapuh jika pesan error berubah — mapping dapat salah klasifikasi	Gunakan typed error atau sentinel error untuk mapping yang lebih robust	Sedang

14. KESIMPULAN

Berdasarkan Code Walkthrough terhadap ketiga layer (Handler, Usecase, Repository), fitur Checkout ACID dinyatakan **LULUS** dengan seluruh skenario black-box (CHK-01 s.d. CHK-08) terpenuhi secara implementasi.

Alur kode telah terbukti konsisten dengan rancangan transaksi checkout:

10. User harus terautentikasi — identitas diambil dari JWT context, bukan body request.
11. Cart kosong ditolak di level usecase sebelum transaksi database dimulai.
12. Stok dicek dan dikurangi dalam satu database transaction (BEGIN / COMMIT / ROLLBACK).
13. Produk dan varian dikunci dengan FOR UPDATE untuk mencegah race condition.
14. Order dibuat dengan status PENDING dan menyimpan snapshot harga (PriceAtPurchase).
15. Cart dikosongkan dalam transaction yang sama — tidak ada cart tersisa setelah order berhasil.
16. Error bisnis dipetakan ke HTTP 400/409/502 — tidak ada unhandled 500.
17. Kegagalan Midtrans tidak membatalkan order internal — order tetap durable.

Dengan demikian, implementasi checkout mendukung seluruh prinsip ACID dan konsisten dengan skenario black-box yang telah diuji sebelumnya. Rekomendasi utama adalah migrasi dari string-matching error menuju typed/sentinel error untuk meningkatkan ketahanan jangka panjang.